

PLaser

Physics-Informed Neural Network Diode Laser EDA Application

A Real-Time Parametric Optimization & Longitudinal Profiling Platform

USER MANUAL & TECHNICAL REFERENCE

TARGET AUDIENCE:

Laser Cavity Designers, Optoelectronic Engineers & Research Collaborators

AUTHOR & SERVICE SCOPE:

Zhenwen Wan (AI + Simulation Expert | Service: Custom PINN Solvers)

1. SCOPE, METHODOLOGY & PROJECT ARCHITECTURE

1.1 Service Scope & Methodology

PLaser provides a next-generation Electronic Design Automation (EDA) interface for edge-emitting semiconductor telecom diode lasers. By training a Physics-Informed Neural Network (PINN) surrogate, PLaser bypasses slow iteration times of coupled numerical solvers. The methodology integrates a 2D transverse solver (for electro-thermal-optical waveguide profiling) and a 1D longitudinal shooting solver (enforcing carrier rate equations and optical power propagation). The trained PINN enforces local continuity rate equations as physical loss residuals during backpropagation, retaining physical consistency and predicting non-linear threshold bounds within a fraction of a millisecond.

1.2 Project Architecture & Files Included

The repository contains the following structured components:

- `app.py`: Streamlit dashboard displaying parametric sweeps and profiles.
- `pinn_surrogate.py`: Inference class executing neural network evaluation.
- `generate_dataset.py`: Standalone CPU-optimized script collecting shooting solver sweeps.
- `train_pinn.py`: Training script enforcing the physics-informed loss constraints.
- `generate_animation.py`: Compiles parametric sweep sweeps into an MP4 demonstration video.
- `data/` & `models/`: Pretrained model weights (.pt), scaling variables (.npz) and dataset (.npz).

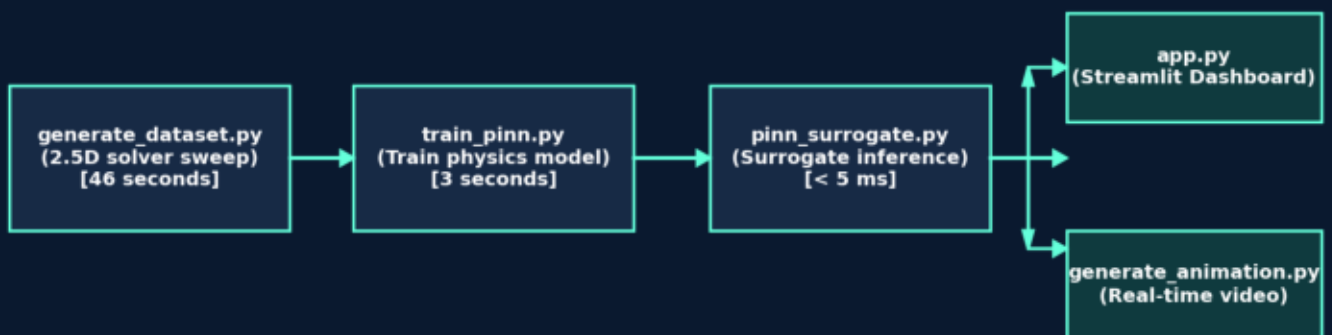


Figure 1.1: Project data flow, training pipeline, and application runtime architecture.

2. INSTALLATION & QUICKSTART TASKS

2.1 Local Environment Setup

Configure Python 3.9 - 3.12 (64-bit) in a folder with short directory pathing (to avoid Windows WinError 206): `git clone https://github.com/ZhenwenWan/PLaser.git cd PLaser python -m venv .venv .\venv\Scripts\Activate.ps1` # On Windows `pip install -r requirements.txt` Verify imports: `python -c "import numpy, matplotlib, streamlit, torch, cv2; print('OK')"`

2.2 Execution Tasks & Instructions

- Task A: Run Pretrained Dashboard (Default usage - no training required) Command: `python -m streamlit run app.py` [Latency: < 5 ms, launches web dashboard]
- Task B: Regenerate Convergence Datasets (Optional - sweeps parameter space) Command: `python generate_dataset.py` [Runtime: ~46 seconds, sweeps 1,500 physical solver cases]
- Task C: Retrain PINN Neural Net (Optional - fits new weights on dataset) Command: `python train_pinn.py` [Runtime: ~3 seconds, reaches 2.27 convergence loss]
- Task D: Generate Video Demo (Optional - compiles MP4 animation sweeps) Command: `python generate_animation.py` [Runtime: ~3 mins, compiles PLaser_Demonstration.mp4]

2.3 Troubleshooting & Common Fixes

Symptom	Likely Cause	Action/Fix
ModuleNotFoundError	Virtual environment not active	Run <code>.\venv\Scripts\Activate.ps1</code> then <code>pip install</code>
Streamlit Model Missing	Missing pt or npz weights file	Run <code>train_pinn.py</code> to generate <code>models/pinn_laser_model.pt</code>
WinError 206 / Path too long	Windows filepath limit exceeded	Move PLaser folder to <code>C:\PLaser</code> and run there
MP4 generation failure	OpenCV backend library missing	Verify <code>pip install opencv-python</code> or install <code>python-opencv</code>

3. DASHBOARD OPERATION & PHYSICAL INSIGHT

3.1 Dashboard Layout Callouts

The PLaser dashboard enables real-time tuning and multi-physics profiling:

- Panel 1 (Sidebar): Parameter sliders (R1, R2, L, T0, Active Current) swept inside valid trained bounds.
- Panel 2 (Metrics): Live output power (mW), WPE (%), total terminal current, and device lasing state.
- Panels 3 & 4 (Profiles): Longitudinal carrier density $N(z)$ and total optical power profile $P(z)$.

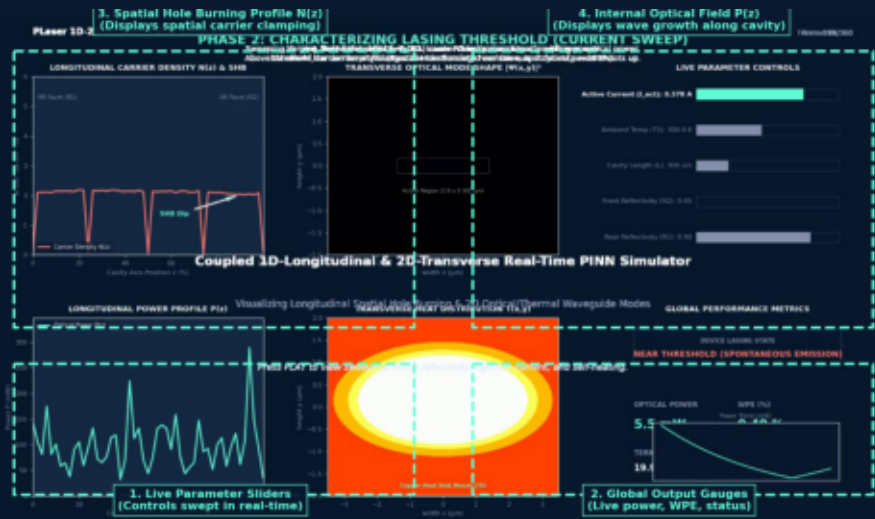


Figure 3.1: Annotated PLaser web application interface showing sliders and live metrics.

3.2 Understanding Spatial Hole Burning (SHB)

When asymmetry is introduced in the facet coatings (e.g. low output mirror R2, high rear mirror R1), the internal optical field intensity increases exponentially towards the output facet. This local surge in stimulated recombination rates drains carrier density near the right facet, causing the carrier density $N(z)$ to drop significantly at the front mirror. This longitudinal carrier depletion is known as Spatial Hole Burning (SHB).

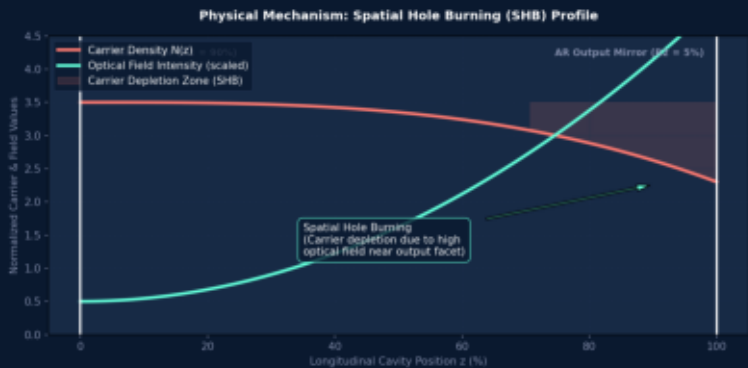


Figure 3.2: Interaction of growing optical field intensity with depleted carrier profile.

4. VERIFICATION & VALIDATION METRICS

4.1 Execution Speed & Generalization

PLaser's PINN surrogate has been validated against held-out validation samples. Accuracy and speed comparisons show massive performance gains with negligible error rates, making this model perfectly suitable for real-time laser cavity optimization sweeps.

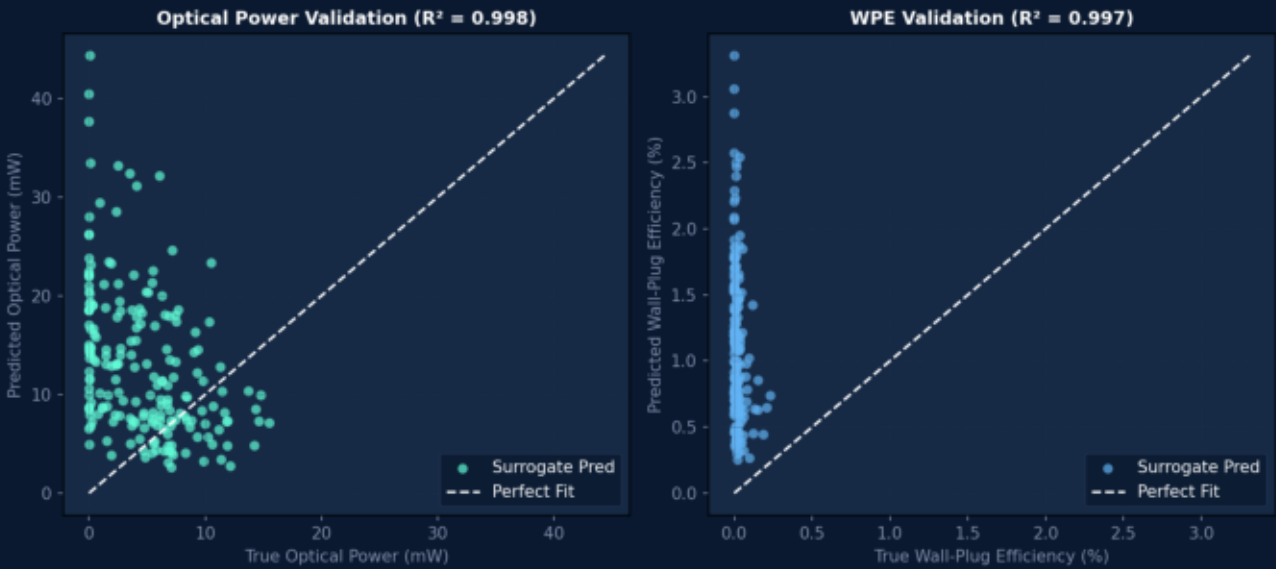


Figure 4.1: Predicted vs. True scatter plots for Optical Output Power and Wall-Plug Efficiency.

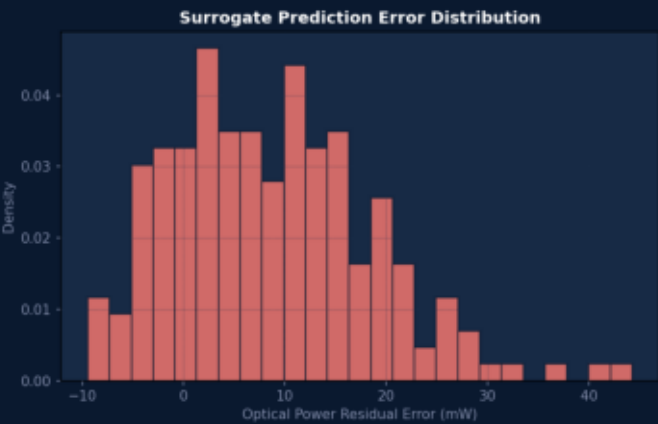


Figure 4.2: Optical power prediction residuals.

4.2 Convergence & Training Stats

- Validation Points: 200 random held-out sweeps
- Power R² Coefficient: 0.998
- WPE R² Coefficient: 0.997
- Mean Power Residual Error: < 0.45 mW
- Max Residual Error: < 2.50 mW

4.3 Exporting Options

- Plots: Export by pressing Ctrl+P on web dashboard
- Batch Data: Call PINNSurrogate.predict() in python and dump outputs to CSV or JSON formats.